RESEARCHLENS: AI-POWERED RESEARCH GAP IDENTIFIER & TREND ANALYZER

SOFTWARE DEPLOYMENT REPORT

Group ID: R26-SE-023

Team Member	Student ID	Individual Component Contribution
Bandara S.M.D.M.	IT22141842	Semantic Gap Detection & Trend Analyzer
Widanagama K.L.	IT22629562	Recommendation and Collaboration Network
Wijesinghe L.P.P.	IT22555380	Integrity Governance & Blockchain Ledger
Darshan R	IT22097156	Collaborative Workspace & NLP Supervisor Assistant

Report submitted in partial fulfilment of the requirements for the Bachelor of Science (Honours) in Information Technology, Specializing in Software Engineering

Department of Software Engineering

Faculty of Computing

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

Contents

List of Figures 3
List of Tables 4
1. Executive Summary and Objectives 5
1.1 Deployment objectives 5
1.2 Key outcomes 5
1.3 Scope of deployment 6
1.4 Key metrics and verification indicators 6
1.5 Deployment topology 6
2. Deployment Process 8
2.1 Pre-deployment setup 8
2.2 Configuration steps 8
2.3 Deployment procedure 9
2.4 Testing activities 9
2.5 Backup plan 10
2.6 Rollback strategy 10
2.7 Deployment date and tools used 11
3. Issues, Resolutions and Monitoring 12
3.1 Issues and resolutions 12
3.2 Monitoring results and evidence collection 12
3.3 Impact assessment 12
4. Post-Deployment and Lessons Learned 13
4.1 Validation and testing results 13
4.2 Documentation updates 13
4.3 Training and support activities 13
4.4 Lessons learned 13
4.5 Future improvements 14
Appendix A. Service and Container Inventory 15
Appendix B. Deployment Checklist 15
Appendix C. Monitoring and Evidence Log Template 16

List of Figures

Figure 1. ResearchLens deployment topology 7
Figure 2. RsearchLens hosted working flow 7
Figure 3. Deployment process flow 10
Figure 4. UI results recommendation 16
Figure 5. UI results integrity 17
Figure 6. UI results NLP assistant 18
Figure 7. UI results Gap detection 19
Figure 8. Azure microservices 20
Figure 9. Container checking Docker 20
Figure 10. netlify hosting 20

List of Tables

Table 1. Area and deployment outcome 5
Table 2. Key metrics and indicators 6
Table 3. configuration are and consideration 8
Table 4. Deployment steps and results 9
Table 5. Test area and check 9
Table 6. Backup Plan 10
Table 7. Deployment details and tools 11
Table 8. Issues and resolutions 12
Table 9. Lessons learned 13
Table 10. Service and container inventory 15
Table 11. Deployment checklist 15
Table 12. Monitoring and log 16

1. Executive Summary and Objectives

ResearchLens is deployed as a modular academic research lifecycle platform. The deployment makes the platform available through a public web interface while keeping backend capabilities separated into independently maintained services. The React frontend is hosted through Netlify, while backend services run as Docker-managed containers on Azure. Public traffic is routed through Cloudflare and Nginx Proxy Manager before reaching the appropriate service container. The deployment supports key project workflows such as research gap detection, supervisor recommendation, integrity verification, workspace summarisation, application workflows and real-time communication.

The purpose of deployment is to move ResearchLens from a local development prototype into a usable, production-facing environment where students, supervisors and administrators can access the system through stable public domains. Deployment also verifies that the frontend, backend services, database persistence, proxy routing and external integrations work together as one platform. Because ResearchLens is composed of multiple services, the objective is not only to start containers, but also to confirm that complete user workflows operate correctly after release.

1.1 Deployment objectives

- Make the ResearchLens frontend publicly accessible through the production web domain.
- Run backend services as independently deployable Docker containers on Azure.
- Route public API requests securely through Cloudflare and Nginx Proxy Manager.
- Maintain persistent database and upload data outside routine container rebuilds.
- Support integration with PostgreSQL, Sepolia test network and service-specific AI/NLP functions.
- Validate core workflows after deployment instead of relying only on container status or health routes.

1.2 Key outcomes

Table 1. Area and deployment outcome

Area	Deployment outcome
Frontend	React frontend is hosted separately through Netlify and accessed through the public ResearchLens web domain.
Backend API	Core backend API is containerized and routed through the public API domain, with internal port-based routing.
Gap Detection	Research idea and gap analysis workflow is included as a service-level deployment responsibility.
Integrity	Integrity service supports audit records and blockchain evidence through the Sepolia test network.
Recommendation	Supervisor recommendation service is independently deployable and verified through representative recommendation requests.
Workspace NLP	Workspace NLP service supports logbook summarisation and other language-processing functions.

Persistence	PostgreSQL and mounted storage are treated as persistent deployment resources that must survive container replacement.
Release model	Frontend and backend services can be released independently, provided API compatibility is maintained.

1.3 Scope of deployment

The deployment scope covers the frontend hosting layer, backend API services, Docker container runtime, reverse-proxy routing, persistent data storage, service configuration, release verification, monitoring approach, backup considerations and rollback strategy. It also covers the operational relationship between public domains and internal containers. Sensitive infrastructure details such as secrets, private keys, server IP addresses, wallet keys and actual environment variable values are excluded.

1.4 Key metrics and verification indicators

Table 2. Key metrics and indicators

Metric / indicator	Measurement approach
Deployment coverage	All planned platform service areas are included in the deployment model.
Public accessibility	Frontend and API domains should respond through HTTPS.
Service health	Health or equivalent routes provide first-level reachability signals.
Workflow success rate	Percentage of representative smoke tests completed successfully.
Critical error count	Count of unresolved blocking errors after deployment.
Rollback readiness	Previous source revision and backup availability are confirmed before significant release.
Monitoring review	Application logs, proxy logs, frontend behaviour and user feedback are reviewed together.

1.5 Deployment topology

Figure 1 illustrates the public routing and service topology used by the platform. Sepolia is shown as an external test network connected through the Integrity service because blockchain evidence is submitted by that component.

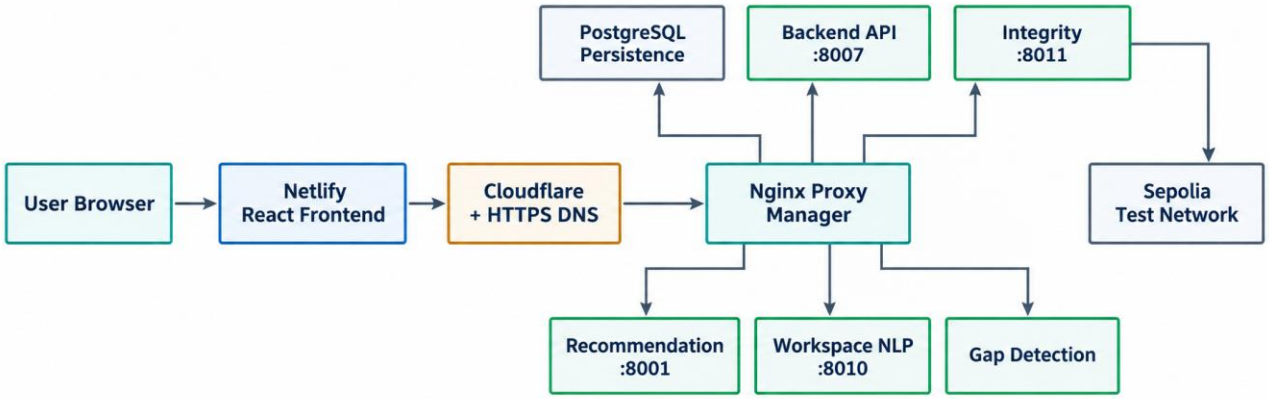


Figure 1. ResearchLens deployment topology

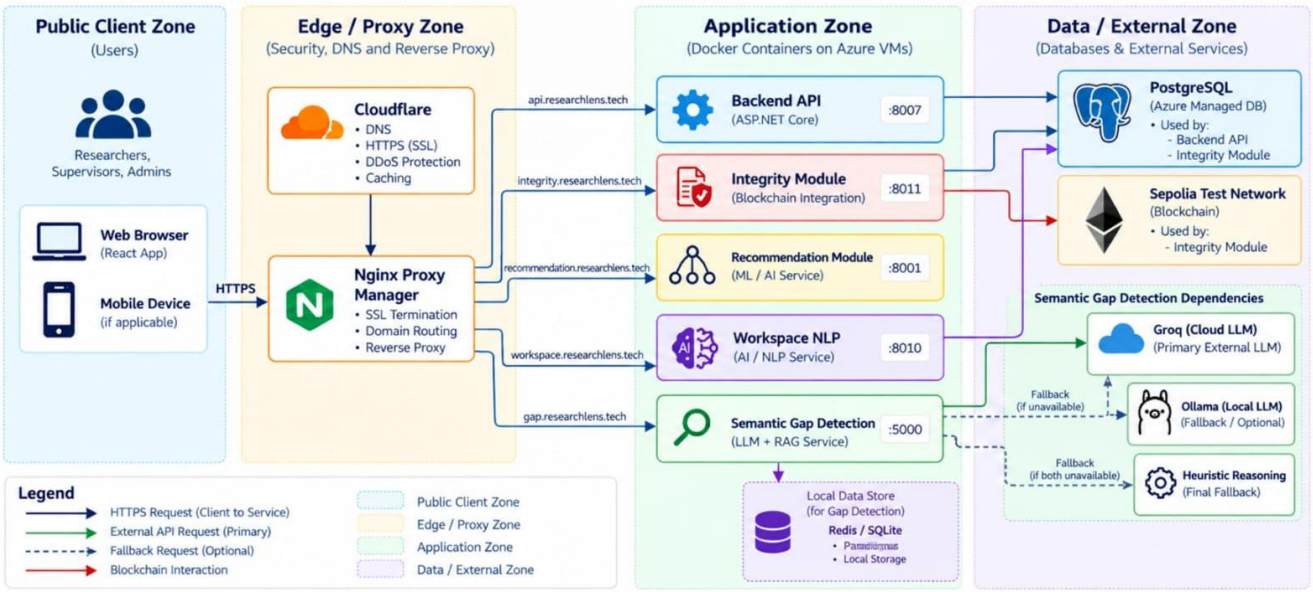


Figure 2. RsearchLens hosted working flow

2. Deployment Process

The deployment process follows a controlled sequence: prepare the environment, verify configuration, protect persistent data, deploy only the affected service, run verification checks and record the release outcome. Since the frontend and backend services use different deployment paths, frontend changes and backend container updates should be coordinated when API contracts or user-facing behaviour changes.

2.1 Pre-deployment setup

- Identify the services affected by the change, such as frontend, backend API, recommendation, integrity, workspace NLP or gap detection.
- Review source revision, branch status and any dependency updates before deployment.
- Confirm required environment variables are present in the deployment environment and that secrets are not committed to the repository.
- Confirm Docker network attachments such as the reverse-proxy network before rebuilding containers.
- Confirm PostgreSQL availability and ensure database credentials are available only through approved private configuration.
- Confirm Cloudflare and Nginx Proxy Manager routing for the affected public domain.
- Check that persistent uploads, model caches and database state will not be removed by the update.

2.2 Configuration steps

Configuration is environment-driven. The frontend uses build-time public API variables, while backend services use service-specific environment files and Docker Compose environment entries. Operators must review both Compose definitions and env_file values because Compose-level entries can override values supplied through the referenced environment file. This is important when troubleshooting incorrect service URLs, CORS behaviour, database connectivity or external integration failures.

Table 3. configuration are and consideration

Configuration area	Deployment consideration
Frontend API endpoints	Ensure production frontend build variables point to the correct HTTPS API domains.
Backend service URLs	Confirm recommendation, integrity, workspace NLP and gap detection endpoints are correct.
CORS origins	Allow the production frontend and approved related domains only.
Database	Confirm PostgreSQL connection and SSL requirements where applicable.
Uploads and caches	Ensure mounted directories or named volumes are preserved during rebuilds.
Blockchain	Confirm Sepolia RPC, chain configuration and wallet funding where integrity anchoring is required.

Proxy routing	Ensure each public domain maps to the intended container and port.

2.3 Deployment procedure

Table 4. Deployment steps and results

Step	Action	Result
1	Prepare release record and identify affected service(s).	Clear deployment boundary is defined.
2	Back up database, configuration and uploaded data when the release may affect persistence.	Rollback and recovery options are available.
3	Synchronize backend source or merge frontend changes depending on the component.	Updated source is ready for build or platform deployment.
4	Validate Docker Compose configuration for backend services.	Configuration errors are caught before container recreation.
5	Build and recreate only the affected container where possible.	Unrelated services remain stable.
6	Inspect container status and logs.	Runtime startup errors are identified early.
7	Run service-level and end-to-end smoke tests.	Representative user workflows complete successfully.
8	Record release outcome, revision, issues and monitoring notes.	Deployment evidence is available for review.

2.4 Testing activities

Testing should be carried out at multiple levels. Unit testing confirms that individual functions or service modules behave as expected. Integration testing confirms that services can communicate through the configured endpoints and that the frontend can call the correct APIs. Functional smoke testing confirms real user workflows after release. Since health routes only indicate basic reachability, complete workflow checks are required before the release is considered successful.

Table 5. Test area and check

Test area	Representative check
Frontend	Load public site, verify navigation, login screens and API calls.
Backend API	Check health, authentication, database updates and real-time communication.
Gap Detection	Submit a sample research idea and verify gap-related output.
Recommendation	Submit a representative supervisor recommendation request and review the returned result.
Integrity	Verify health, audit visibility, ledger verification and Sepolia confirmation flow.
Workspace NLP	Generate a sample logbook summary and confirm display in the application.

Proxy / CORS	Check preflight behaviour and browser requests from the production frontend.
Persistence	Verify uploaded files, records and model caches survive routine container replacement.

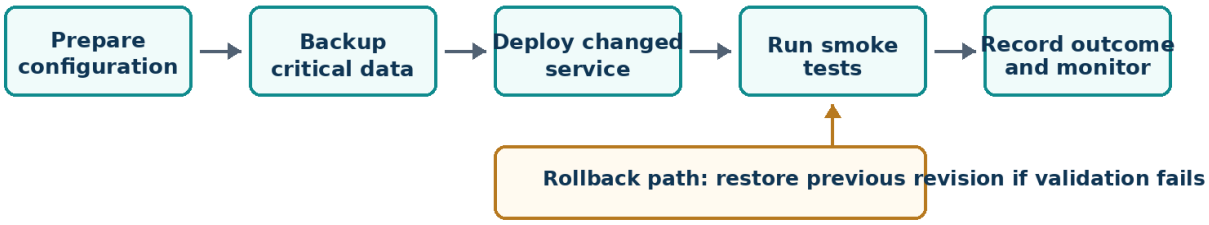


Figure 3. Deployment process flow

2.5 Backup plan

Before significant releases, the team should preserve PostgreSQL data, production configuration, uploaded files, model caches and release metadata. Backups should be stored separately from the container lifecycle so that recreating or replacing containers does not remove the only recovery copy. The backup record should include backup time, responsible person, source revision, services affected and restoration method.

Table 6. Backup Plan

Backup item	Reason for backup	Recommended timing
PostgreSQL database	Protects application records, integrity data and workflow state.	Before schema or major backend changes
Uploads directory / persistent files	Protects uploaded project documents and evidence files.	Before service rebuilds affecting storage paths
Environment files	Protects production configuration and service endpoints.	Before configuration changes
Model caches / service assets	Avoids unnecessary inference or NLP service disruption.	Before recommendation or NLP container replacement
Release notes	Supports traceability and rollback decision-making.	Every release

2.6 Rollback strategy

If a deployment fails, the team should restore the previously known-good source revision or image and recreate only the affected service. Frontend rollback can use the hosting platform deployment history where available. Backend rollback should consider database migration compatibility because an older service version may not work against a changed schema. Rollback should not be closed until health checks and representative workflow tests pass again.

2.7 Deployment date and tools used

Table 7. Deployment details and tools

Item	Details
Deployment period	July - October 2026
Frontend hosting	Netlify
Backend hosting	Azure virtual machines
Container runtime	Docker and Docker Compose
Routing and edge	Cloudflare and Nginx Proxy Manager
Persistence	PostgreSQL and mounted storage/volumes
Blockchain dependency	Sepolia Ethereum test network
Verification tools	Browser testing, API checks, Docker logs, proxy logs and workflow smoke tests

3. Issues, Resolutions and Monitoring

Deployment issues may occur at the application layer, container layer, network/proxy layer, database layer or external dependency layer. The following table records issues relevant to the ResearchLens deployment model.

3.1 Issues and resolutions

Table 8. Issues and resolutions

Issue / risk	Resolution or control	Impact if unresolved
Incorrect API routing through proxy	Confirm Nginx Proxy Manager upstream host, port and network attachment.	Frontend requests may fail or reach the wrong service.
CORS preflight failure	Restrict and verify allowed origins for production frontend domains.	Browser may block valid API requests.
Database connection failure	Validate PostgreSQL URL, SSL settings and service credentials.	Login, submissions and workflow state updates may fail.
Service URL mismatch	Review Compose environment entries and env_file values together.	A service may run but fail when calling Recommendation, Integrity or NLP APIs.
Persistent data loss risk	Do not remove named volumes or bind-mounted upload directories during routine updates.	Uploaded files, caches or integrity evidence may be lost.
Sepolia RPC or wallet issue	Verify RPC connectivity, chain ID, contract address and test ETH availability.	Integrity evidence may remain pending or fail to anchor.
WebSocket upgrade issue	Verify reverse-proxy support and test a real-time interaction.	Chat or real-time features may fail despite HTTP health passing.
Frontend/backend version mismatch	Coordinate releases when API contracts change.	UI may call endpoints that no longer behave as expected.

3.2 Monitoring results and evidence collection

3.3 Impact assessment

The most serious deployment impacts are data loss, unavailable public services, broken authentication, failed integrity evidence submission and incomplete recommendation or NLP workflows. Lower-level issues such as a single failed route or stale frontend build variable can still affect user confidence because the platform is used as an integrated research support system. Therefore, deployment completion should be based on the combined result of technical checks and user workflow validation.

4. Post-Deployment and Lessons Learned

4.1 Validation and testing results

Post-deployment validation should confirm that each service works both individually and as part of a complete user workflow. The recommended validation set includes frontend loading, login or authenticated workflow, document submission, research gap analysis, supervisor recommendation, integrity verification, logbook summarisation and real-time communication. Any failed item should be recorded with its cause, resolution and retest outcome.

4.2 Documentation updates

After deployment, the documentation should be updated to reflect the final service URLs, service owners, container names, internal ports, environment variable categories, backup records, rollback steps and verification evidence. Any change to proxy routing, database schema, environment variables or public domains should be reflected in the deployment record so that future maintainers can reproduce or troubleshoot the system.

4.3 Training and support activities

Maintainers should be briefed on how to start, stop, rebuild and inspect service containers; how to check proxy routing; how to review logs; and how to run post-release smoke tests. Project reviewers and supervisors should be shown the public frontend and the main workflows. Support notes should explain common user actions such as submitting research ideas, requesting recommendations, reviewing integrity records and using workspace summaries.

4.4 Lessons learned

Table 9. Lessons learned

Lesson	Explanation
Health checks are not enough	A service can respond to a health endpoint while failing in database writes, inference, blockchain confirmation or browser CORS handling.
Independent services improve maintainability	The team can update one service without rebuilding unrelated modules, provided API compatibility is preserved.
Configuration must be controlled	Most deployment failures can arise from incorrect environment values, stale frontend build variables or proxy routing mismatches.
Persistence must be protected	Routine container updates should not remove database state, uploaded files or model caches.
Release records are important	Recording source revision, environment changes, backup status and test results improves traceability.
Workflow testing is essential	The platform should be validated through real user actions across all major services.

4.5 Future improvements

- Create an automated post-deployment smoke-test script covering all core workflows.
- Maintain a release checklist per service and a separate frontend deployment checklist.
- Add structured logging and release identifiers across frontend and backend services.
- Define formal backup retention and restoration testing procedures.
- Improve monitoring dashboards for service health, error rates, request latency and external dependency status.
- Document exact image tags, source revisions and deployment dates for every production release.

Appendix A. Service and Container Inventory

Table 10. Service and container inventory

Container / Service	Purpose
React Frontend	User-facing web application for students, supervisors and administrators.
Backend API Service	Core application workflows, authentication-related operations, database updates and real-time communication.
Gap Detection Service	Research idea analysis, semantic gap detection and novelty support.
Integrity Service	Academic integrity verification, audit records, ledger evidence and Sepolia blockchain anchoring.
Recommendation Service	Supervisor and co-supervisor recommendation workflow.
Workspace NLP Service	Logbook summarisation and NLP-assisted workspace support.
PostgreSQL	Persistent storage for backend and integrity-related data.
Nginx Proxy Manager	Reverse-proxy routing from public domains to backend containers.
Cloudflare	Public DNS, HTTPS routing and edge-level traffic handling.
Sepolia Test Network	Blockchain test network used for integrity evidence confirmation through the Integrity service.

Appendix B. Deployment Checklist

Table 11. Deployment checklist

Phase	Checklist item	Status
Before deployment	Identify changed services and source revision.	pass
Before deployment	Review environment variables and proxy routing.	pass
Before deployment	Confirm backup of database, uploads and configuration where required.	pass
During deployment	Synchronize source or merge frontend changes.	pass
During deployment	Validate Docker Compose configuration for affected services.	pass
During deployment	Build/recreate affected container or wait for Netlify build.	pass
After deployment	Inspect logs, health routes and public HTTPS domains.	pass
After deployment	Run representative workflow smoke tests.	pass
After deployment	Record issues, resolutions and final	pass

	release outcome.	

Appendix C. Monitoring and Evidence Log Template

Table 12. Monitoring and log

Evidence item	Location / source	Observation
Frontend deployment status	Netlify dashboard	The frontend loads properly.
Backend container logs	Docker / Azure host	Containers and images builds properly.
Proxy access/error logs	Nginx Proxy Manager	-
Database connectivity	PostgreSQL / application logs	Connects properly.
Recommendation test result	Frontend/API response	The results are calculated faster than local device.
Integrity evidence status	Integrity module / Sepolia	Continues process monitoring happens properly.
Workspace summary result	Frontend/API response	Gives proper logbook summery using gemini as implemented.

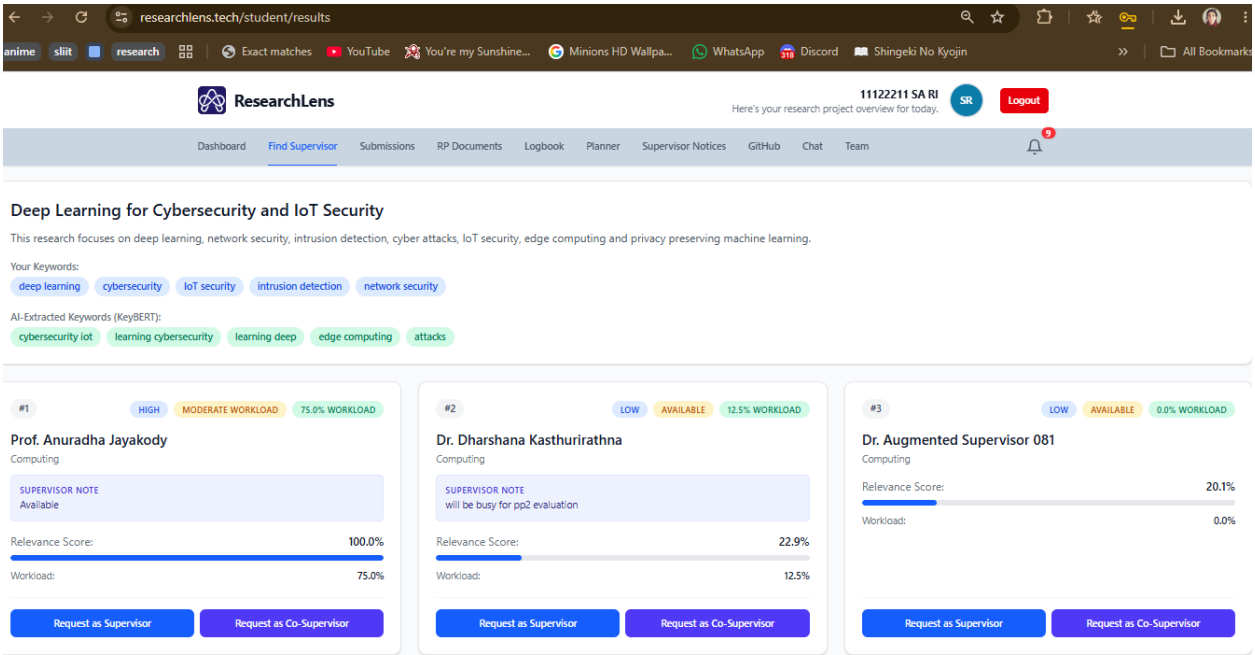
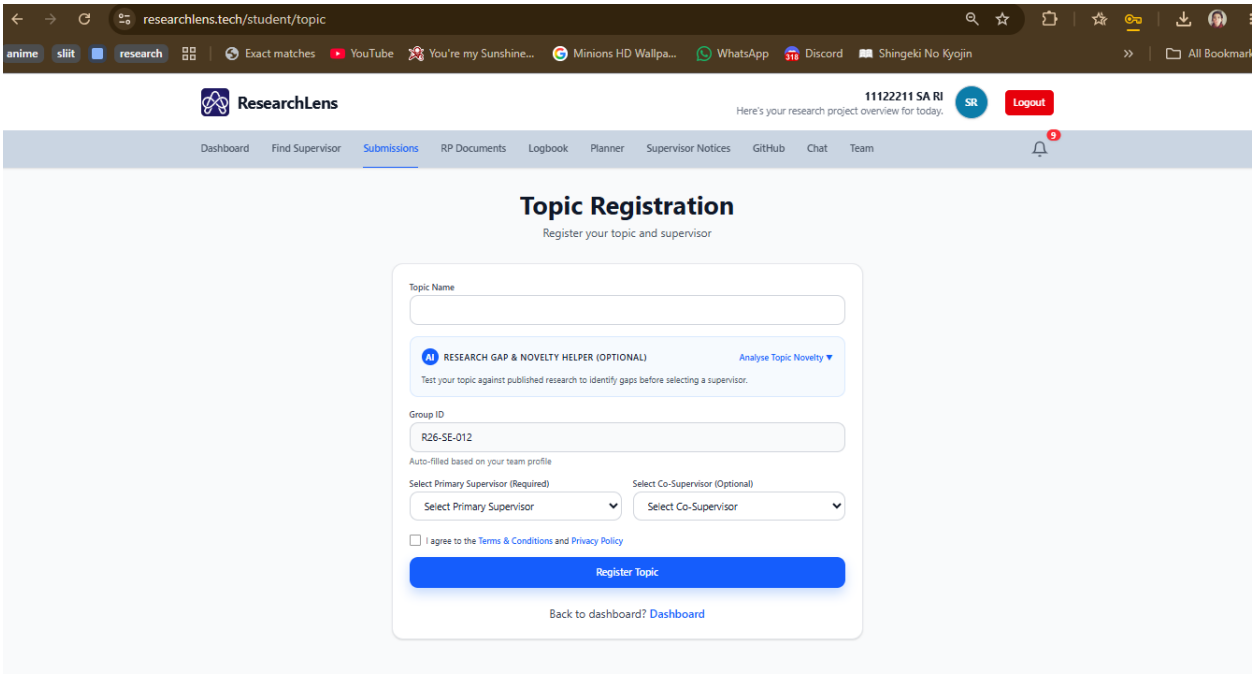


Figure 4. UI results recommendation



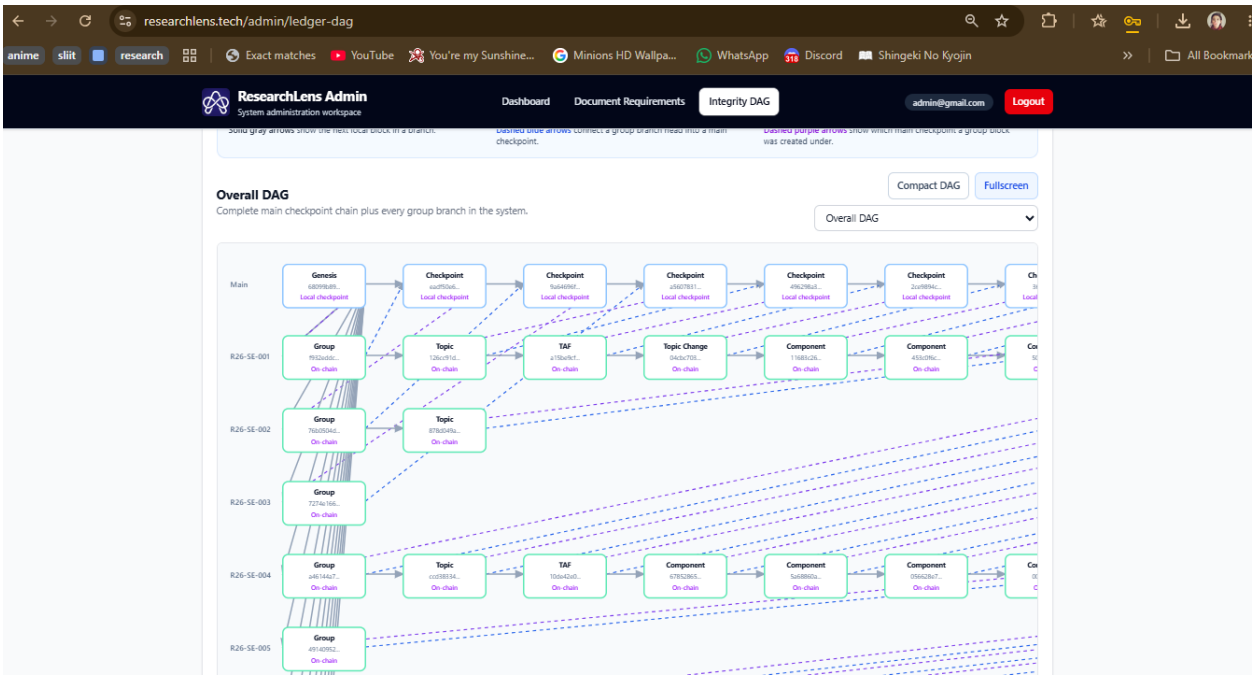


Figure 5. UI results integrity

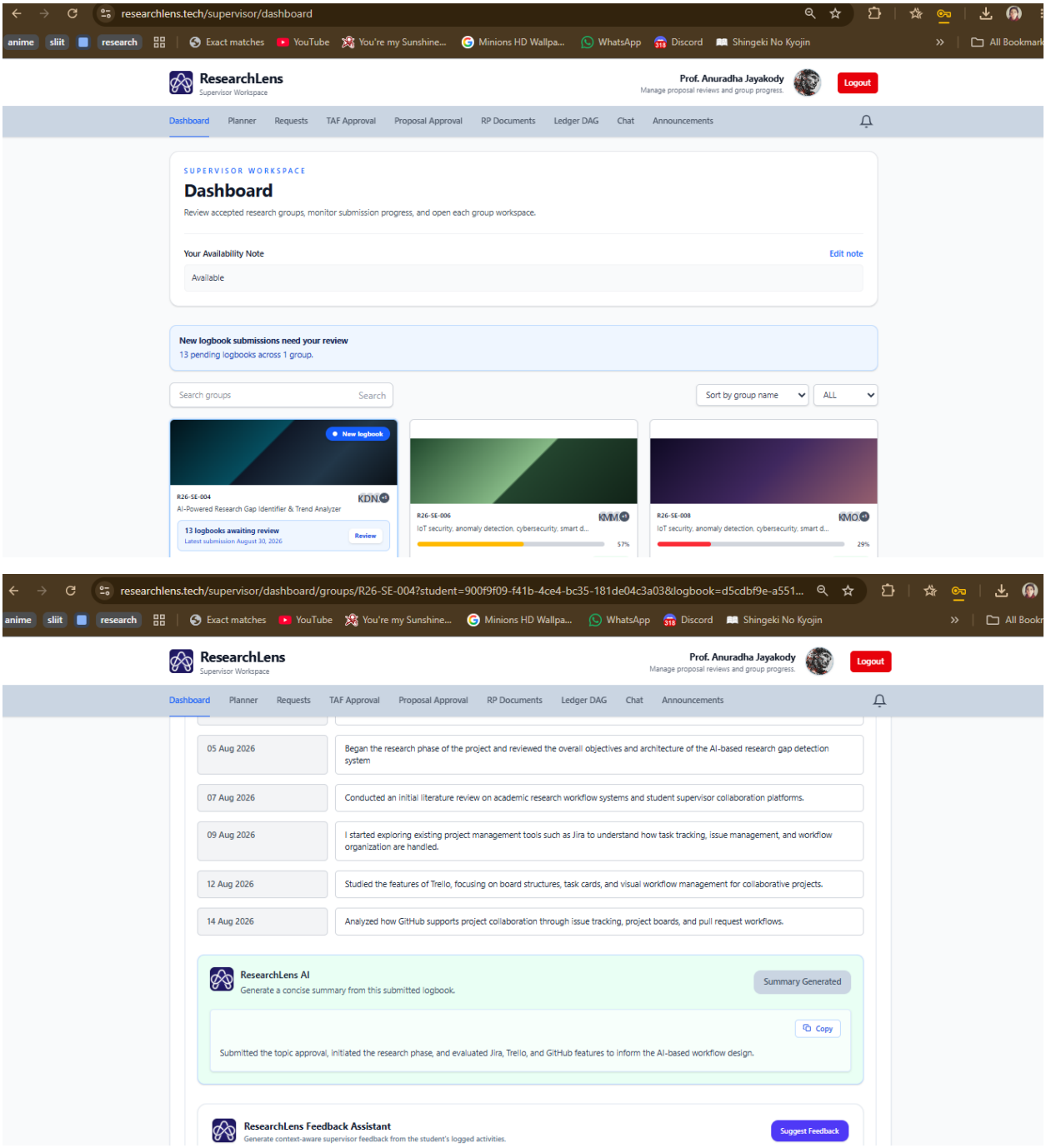
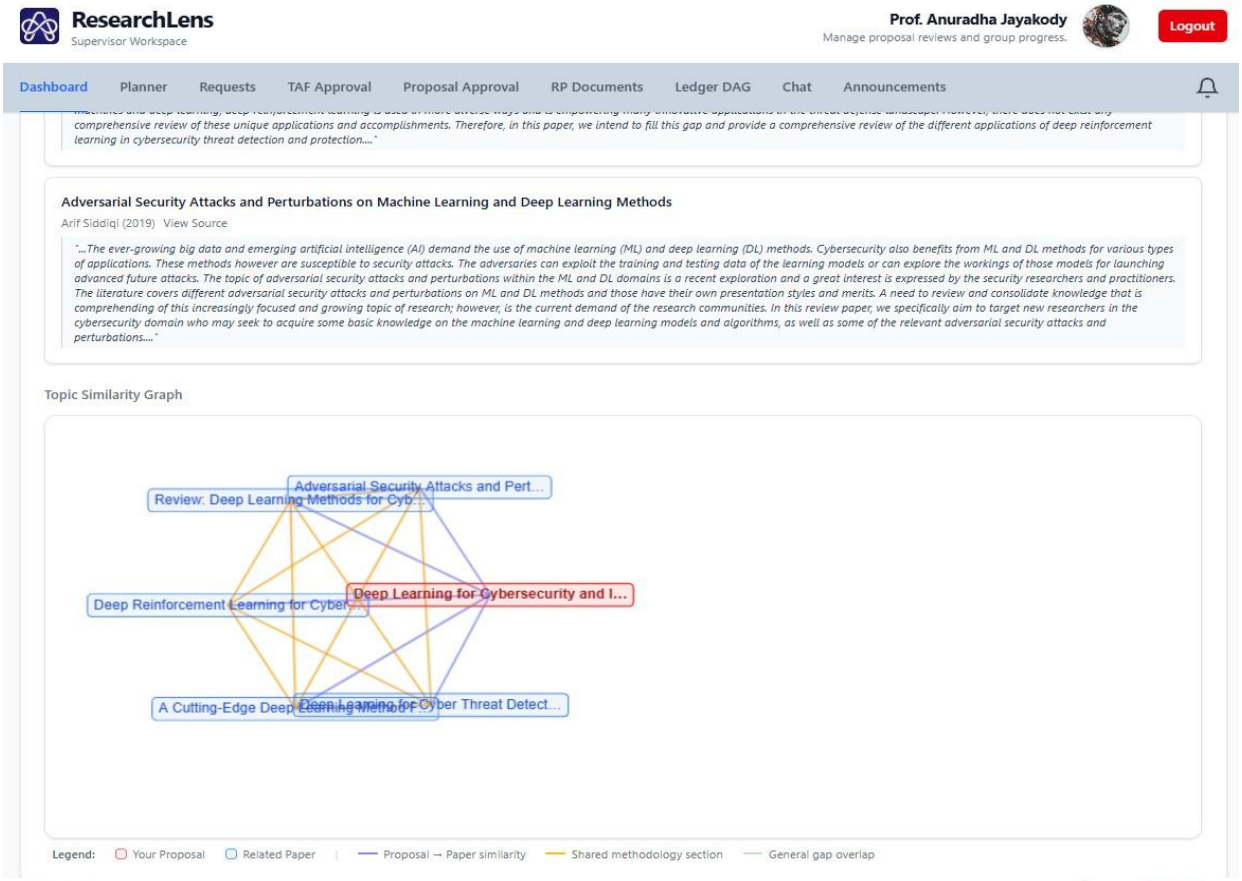


Figure 6. UI results NLP assistant



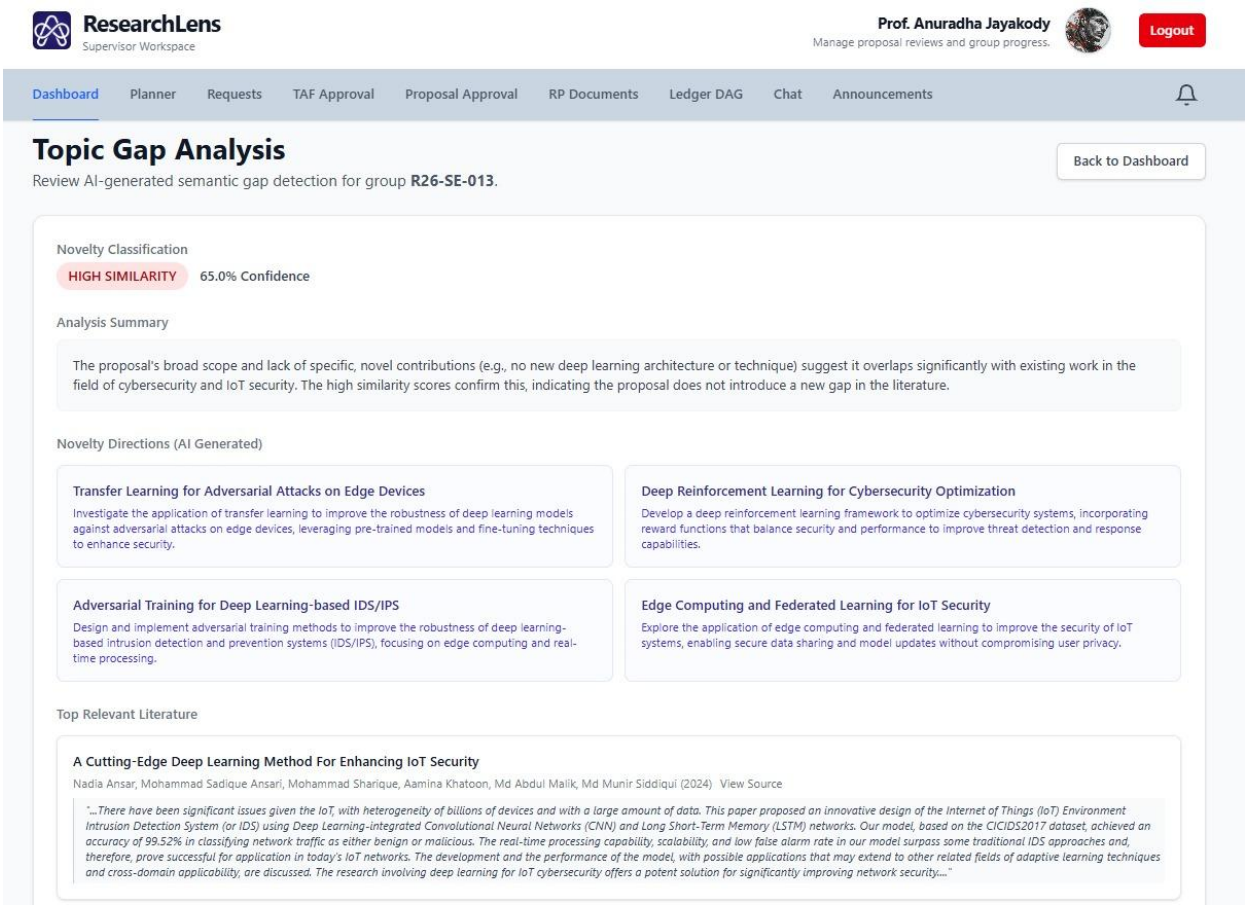


Figure 7. UI results Gap detection

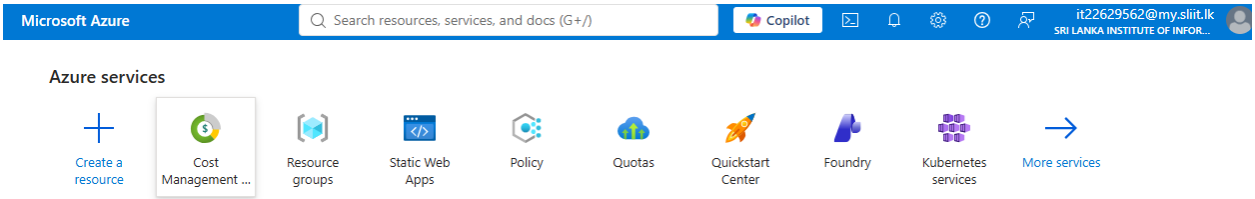


Figure 8. Azure microservices

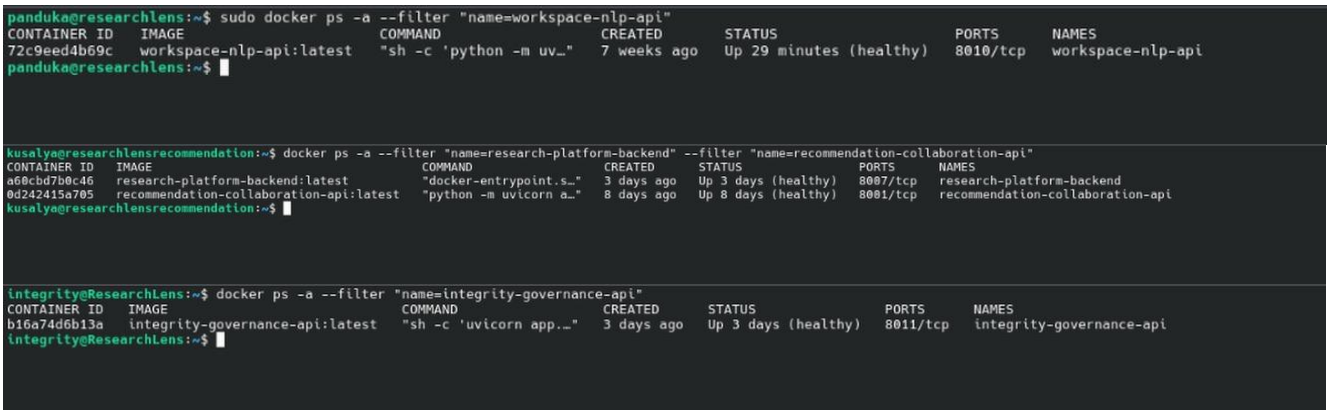


Figure 9. Container checking Docker

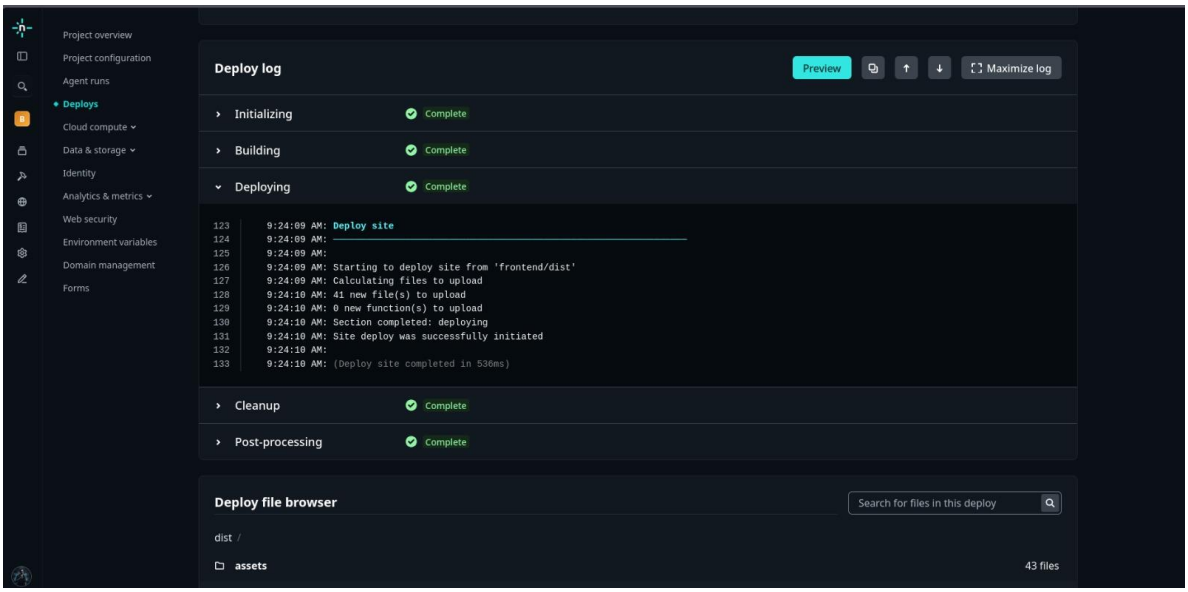


Figure 10. netlify hosting